

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..12

CUSTOMIZATIONS

 Copy as Markdown

Customize Apple sign-in

This page covers additional configuration options for the Apple identity provider beyond the basic setup. On the server, you can control how credentials are loaded, react to account creation, and configure the web routes. In your app, you can configure Sign in with Apple, customize the sign-in button with the `AppleSignInWidget`, or build a completely custom interface with the `AppleAuthController`.

Server configuration

Below is a non-exhaustive list of some of the most common configuration options. For more details on all options, check the `AppleIdpConfig` in-code documentation.

Loading Apple credentials

The `AppleIdpConfigFromPasswords()` constructor reads the eight `apple*` keys from `config/passwords.yaml` (or the matching `SERVERPOD_PASSWORD_` environment variables) for you. This is the path used in the [setup guide](#) and is the recommended default:

```
final appleIdpConfig = AppleIdpConfigFromPasswords();
```

Use `AppleIdpConfig(...)` directly when you need to pull credentials from a custom source, transform them at startup, or omit `passwords.yaml` entirely. You are responsible for resolving each value:

```
final appleIdpConfig = AppleIdpConfig(  
  serviceIdentifier: pod.getPassword('appleServiceIdentifier')!,  
  bundleIdentifier: pod.getPassword('appleBundleIdentifier')!,  
  redirectUri: pod.getPassword('appleRedirectUri')!,  
  teamId: pod.getPassword('appleTeamId')!,  
  keyId: pod.getPassword('appleKeyId')!,  
  key: pod.getPassword('appleKey')!,  
  webRedirectUri: pod.getPassword('appleWebRedirectUri'),  
  androidPackageIdentifier: pod.getPassword('appleAndroidPackageIdentifier'),
```



```
);
```

Reacting to account creation

For Apple-specific logic, use `onAfterAppleAccountCreated` on `AppleIdpConfig`. It receives the created `AppleAccount` row, so you can read the Apple identifier and the name Apple returned on first sign-in.

```
AppleIdpConfigFromPasswords(  
  onAfterAppleAccountCreated:  
    (session, authUser, appleAccount, {required transaction}) async {  
      session.log('Apple account created: ${appleAccount.userIdentifier}');  
    },  
)
```

For logic that should run whichever provider the user signed in with, use `onAfterAuthUserCreated` on `AuthUsersConfig` instead. It fires the first time any provider creates an auth user.

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    AppleIdpConfigFromPasswords(),  
  ],  
  authUsersConfig: AuthUsersConfig(  
    onAfterAuthUserCreated: (session, authUser, {required transaction}) async {  
      // authUser.id is the new user's UUID, use it to create any  
      // app-specific records that must exist before the user's first request.  
      await UserData.db.insertRow(  
        session,  
        UserData(authUserId: authUser.id, createdAt: authUser.createdAt),  
        transaction:  
      );  
    },  
  ),  
);
```

INFO

This callback runs inside the same database transaction as the auth user creation. Throwing an exception inside it will abort the entire process and the user will not be created. If you perform external side-effects (e.g. analytics, sending emails), wrap them in a try/catch so an unrelated failure does not block sign-in.

Web routes configuration

Sign in with Apple requires web routes for handling callbacks and notifications. These routes must be configured both on Apple's side and in your Serverpod server, using the

`pod.configureAppleIdpRoutes()` method:

```
pod.configureAppleIdpRoutes(  
  revokedNotificationRoutePath: '/hooks/apple-notification',  
  webAuthenticationCallbackRoutePath: '/auth/callback',  
);
```

- `revokedNotificationRoutePath` (default: `'/hooks/apple-notification'`): The path Apple calls when a user revokes authorization. Register this URL in your Apple Developer Portal for server-to-server notifications.
- `webAuthenticationCallbackRoutePath` (default: `'/auth/apple/callback'`): The path Apple redirects to after the user completes web-based sign-in. Must match the return URL registered on your Service ID.

NOTE

When a user revokes access from their Apple ID settings, Apple sends a notification to `revokedNotificationRoutePath`. Registering the route is enough: Serverpod revokes the Apple authorization and the tokens it issued through Apple sign-in for that user. Clean up only your own derived records.

AppleIdpConfig parameters

Parameter	Type	Required	password.s.yaml key	Description
<code>serviceIdentifier</code>	String	Yes	<code>appleServiceIdentifier</code>	The Services ID identifier (e.g. <code>com.example.service</code>). Required on every platform, though only the Android and web OAuth flow uses it.
<code>bundleIdentifier</code>	String	Yes	<code>appleBundleIdentifier</code>	The App ID bundle identifier (e.g. <code>com.example.app</code>). Used as the client ID for native Apple platform sign-in.
<code>redirectUri</code>	String	Yes	<code>appleRedirectUri</code>	The server callback route Apple redirects to after sign-in. Sent with every authorization-code exchange, and validated by Apple in the Android and web flow. Must be HTTPS and match the return URL registered on your Service ID.

Parameter	Type	Required	password s.yaml key	Description
teamId	String	Yes	appleTeamId	The 10-character Team ID from your Apple Developer account. Used to sign the client secret JWT.
keyId	String	Yes	appleKeyId	The Key ID of the Sign in with Apple private key.
key	String	Yes	appleKeyy	The raw contents of the .p8 private key file, including the -----BEGIN PRIVATE KEY----- header and footer. Do not pre-generate the JWT yourself.
webRedirectUri	String?	web only	appleWebRedirectUri	The web app URL the browser is redirected to after the server receives Apple's callback.
androidPackageIdentifier	String?	Android only	appleAndroidPackageIdentifier	The Android package name (e.g. com.example.app). When set, the callback route redirects Android sign-ins back to the app via an intent URI.

App configuration

These options control the values your app passes to `initializeAppleSignIn()`.

Configuring Sign in with Apple on the app

Your app needs the Service ID and the server callback URL. The setup guide passes them via `--dart-define`. If you would rather hardcode them or resolve them at runtime, pass them directly to `initializeAppleSignIn()` instead:

```
client.auth.initializeAppleSignIn(
  serviceIdentifier: 'com.example.service',
  redirectUri: 'https://example.com/auth/callback',
);
```

When both are passed, they take precedence over the `APPLE_SERVICE_IDENTIFIER` and `APPLE_REDIRECT_URI` build variables. The `redirectUri` must match the **Return URL** registered on your Apple Service ID and the value used by `pod.configureAppleIdpRoutes()`.

NOTE

Only the web and Android flow consumes these values, but `initializeAppleSignIn` requires them on every platform. Pass them (or the matching dart-defines) even in an iOS-only app, or initialization throws.

Using environment variables

The build variables `APPLE_SERVICE_IDENTIFIER` and `APPLE_REDIRECT_URI` are read by `initializeAppleSignIn()` whenever you do not pass the values as parameters:

- `APPLE_SERVICE_IDENTIFIER`: your Services ID identifier (e.g. `com.example.service`)
- `APPLE_REDIRECT_URI`: the server callback URL (e.g. `https://example.com/auth/callback`)

Pass them at build time with `--dart-define`:

```
flutter run \  
  --dart-define="APPLE_SERVICE_IDENTIFIER=com.example.service" \  
  --dart-define="APPLE_REDIRECT_URI=https://example.com/auth/callback"
```

This approach is useful when you need to:

- Manage configuration separately for different platforms (Android, web) in a centralized way
- Avoid committing sensitive configuration to version control
- Configure different credentials for different build environments (development, staging, production)

TIP

You can set `--dart-define` values in your IDE run configuration or CI/CD pipeline instead of passing them on every `flutter run` command.

Customize the sign-in button

For the `buttonStyle` precedence rules that apply to all built-in buttons, see [Styling the buttons](#).

INFO

The `SignInWidget` uses the `AppleSignInWidget` internally to display the Apple sign-in flow. You can also supply a custom `AppleSignInWidget` to the `SignInWidget` to override the default behavior.

```
SignInWidget(  
  client: client,  
  appleSignInWidget: AppleSignInWidget(  
    client: client,  
    // Shape and label survive inside SignInWidget, unless its buttonStyle  
    // sets them. Brand colors do not.  
    shape: SignInButtonShape.rounded,  
    text: SignInButtonTextVariant.signInWith,  
    // A custom widget replaces the built-in handling, so pass your own callbacks.  
    onAuthenticated: () { /* ... */ },  
    onError: (error) { /* ... */ },  
  ),  
)
```

Using the `AppleSignInWidget`

The `AppleSignInWidget` handles the complete Apple sign-in flow for iOS, macOS, Android, and web.

You can customize the widget's appearance and behavior:

```
// AppleIDAuthorizationScopes comes from the sign_in_with_apple package.  
// Add it to your app's dependencies to import it.  
import 'package:sign_in_with_apple/sign_in_with_apple.dart';  
  
AppleSignInWidget(  
  client: client,  
  // Button customization. The values shown are the defaults.  
  style: AppleButtonStyle.black, // or white, whiteOutlined  
  size: SignInButtonSize.large, // or medium, small  
  text: SignInButtonTextVariant.continueWith, // or signInWith, signUpWith, signIn  
  shape: SignInButtonShape.pill, // or rounded, rectangular  
  logoAlignment: SignInButtonLogoAlignment.center, // or left  
  minimumWidth: 240, // at most 400  
  textStyle: null, // TextStyle for the label  
  
  // Scopes to request from Apple.  
  // These are the default, and the only ones Sign in with Apple supports.  
  scopes: const [  
    AppleIDAuthorizationScopes.email,  
    AppleIDAuthorizationScopes.fullName,  
  ],  
  
  onAuthenticated: () {  
    // Do something when the user is authenticated.  
    //  
    // NOTE: You should not navigate to the home screen here, otherwise  
    // the user will have to sign in again every time they open the app.  
  },  
)
```

```
    onError: (error) {  
      // Handle errors  
    },  
  )  
}
```

Build a custom UI with AppleAuthController

For more control over the UI, you can use the `AppleAuthController` class, which provides all the authentication logic without any UI components. This allows you to build a completely custom authentication interface.

```
import 'package:serverpod_auth_idp_flutter/serverpod_auth_idp_flutter.dart';  
  
// Also import sign_in_with_apple here for AppleIDAuthorizationScopes.  
final controller = AppleAuthController(  
  client: client,  
  onAuthenticated: () {  
    // Do something when the user is authenticated.  
    //  
    // NOTE: You should not navigate to the home screen here, otherwise  
    // the user will have to sign in again every time they open the app.  
  },  
  onError: (error) {  
    // Handle errors  
  },  
  scopes: const [  
    AppleIDAuthorizationScopes.email,  
    AppleIDAuthorizationScopes.fullName,  
  ],  
);  
  
// Initiate sign-in  
await controller.signIn();
```

AppleAuthController state management

Your widget should render the appropriate UI based on the `state` property of the controller. You can also use the below state properties to build your UI:

```
// Check current state  
final state = controller.state; // AppleAuthState enum  
  
// Check if loading  
final isLoading = controller.isLoading;  
  
// Check if authenticated  
final isAuthenticated = controller.isAuthenticated;  
  
// Get error message  
final errorMessage = controller.errorMessage;
```

```
// Listen to state changes
controller.addListener(() {
  setState(() {
    // Rebuild UI when controller state changes
  });
});
```

AppleAuthController states

- `AppleAuthState.idle` - Ready for user interaction.
- `AppleAuthState.loading` - Processing a sign-in request.
- `AppleAuthState.error` - An error occurred.
- `AppleAuthState.authenticated` - Authentication was successful.

Related

- [Setup](#): configure Sign in with Apple on the server and in your app.
- [Troubleshooting](#): fix common Apple sign-in errors.
- [UI components](#): style the sign-in buttons and localize the built-in UI.
- [Working with users](#): manage auth users and react to account events.